

Scaling Rules for Optimization

Jeremy Bernstein
jbernstein@mit.edub



Plan for today

introduce the optimization problem

cover some classical approaches

- Newton
- Gauss-Newton
- steepest descent

heuristic picture of scaling

modularizing the theory

width

depth

The machine learning puzzle

Three pieces to the puzzle:

- ① Approximation Does there exist a neural net in my model family that fits the training data?
- ② Optimization If it does exist, can I find it?
- ③ Generalization Does it work well on unseen data?

This lecture will focus mainly on the second question.

The optimization problem: formal statement

neural net $f(x, w)$ prediction

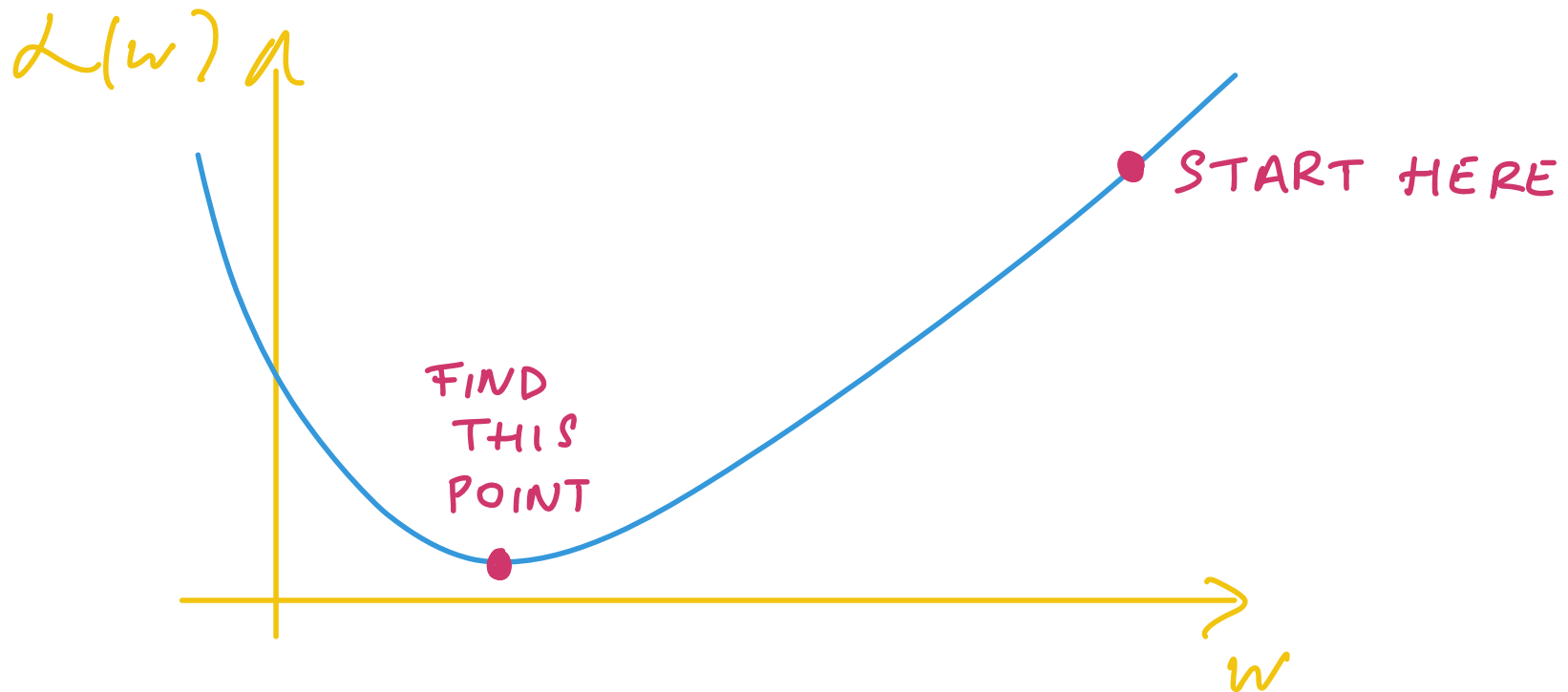
error measure $\mathcal{L}(\hat{y}, y)$ target

training data $(x^{(1)}, y^{(1)}), \dots, (x^{(N)}, y^{(N)})$

loss function $\mathcal{L}(w) = \frac{1}{N} \sum_{i=1}^N \mathcal{L}(f(x^{(i)}, w), y^{(i)})$

GOAL find w that minimizes $\mathcal{L}(w)$

The optimization problem: a picture



roughly, we just iterate

$$w \rightarrow w - \eta \frac{\partial L}{\partial w}$$

learning rate⁵

gradient

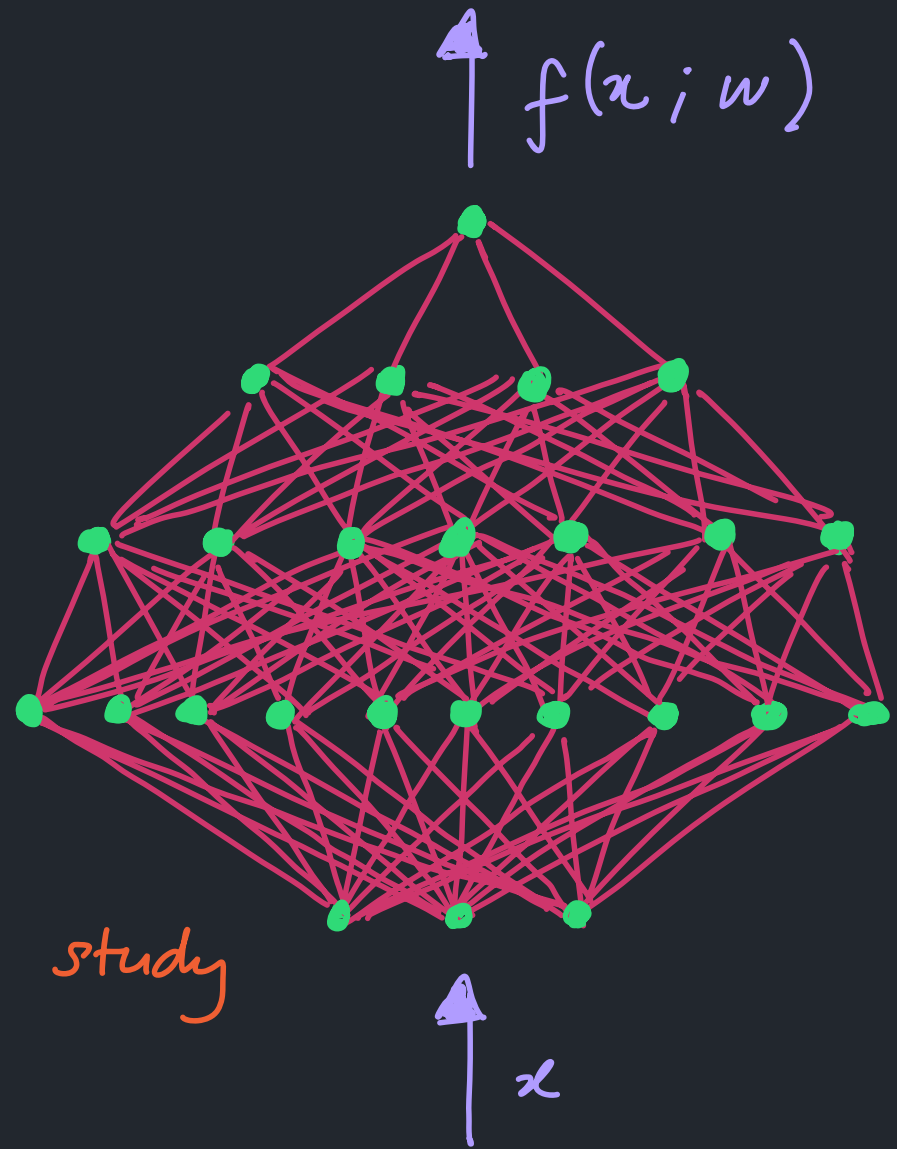
What makes optimization hard?

Some examples:

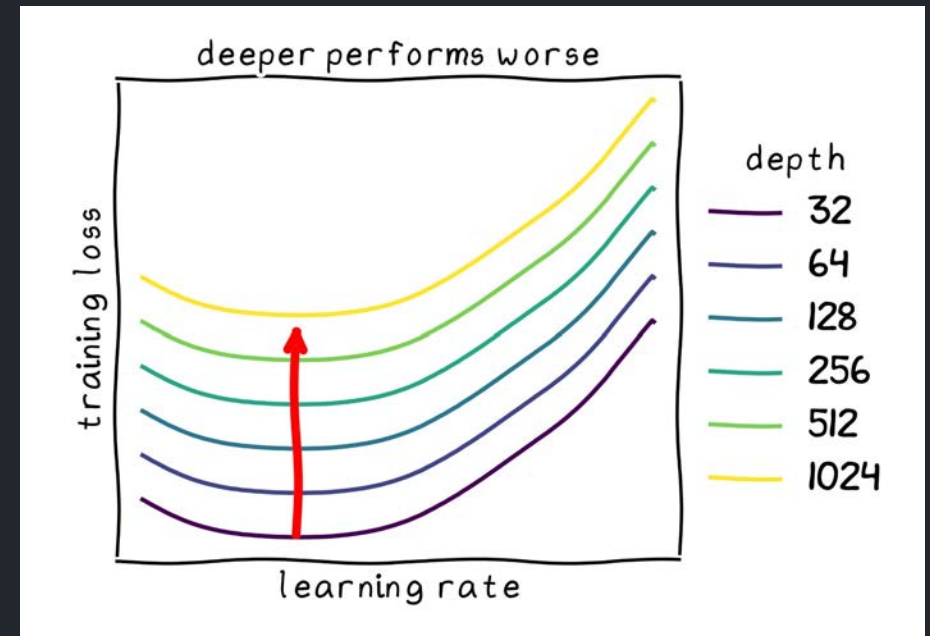
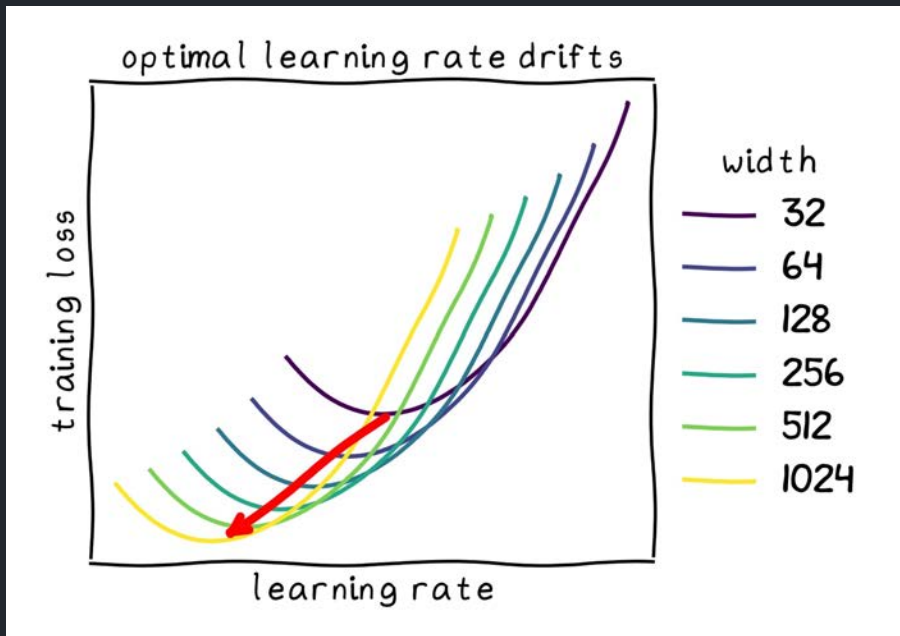
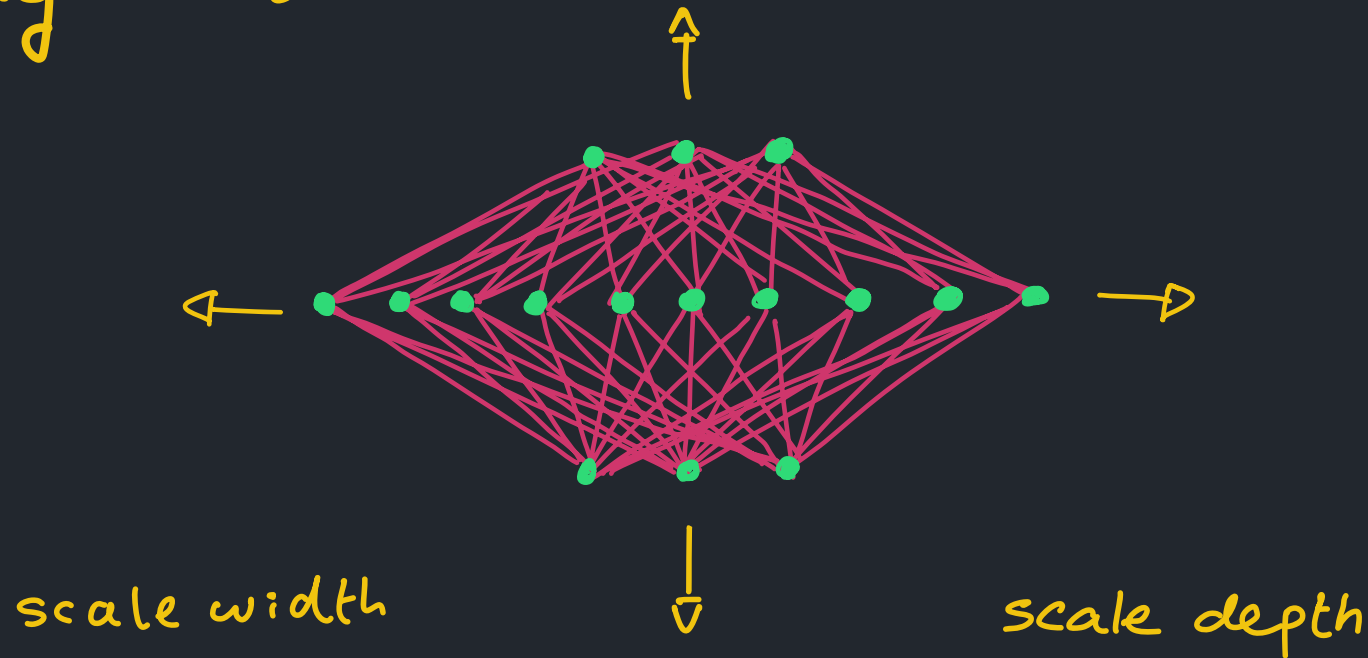
- ① size: a lot of weights
 - ② depth: a lot of layers
 - ③ noise: a lot of data
- ⇓
must use
mini-batches

In this lecture, we will just study
full-batch optimization...

... it's already interesting.



Scaling woes



Classical 1st and 2nd Order Methods

Let's survey some optimization methods

We will look at:

- first-order methods — use 1st derivatives

$$\text{gradient } g = \frac{\partial \mathcal{L}}{\partial w}$$

- second-order methods — use 2nd derivatives

$$\text{Hessian } H = \frac{\partial^2 \mathcal{L}}{\partial w^2}$$

Will try to highlight modelling assumptions
and potential downsides of the different methods

Taylor expanding the loss

Different classical approaches to optimization take this Taylor expansion as a starting point

$$\begin{aligned} \mathcal{L}(w + \Delta w) &= \mathcal{L}(w) + \underbrace{\frac{\partial \mathcal{L}}{\partial w}^T \Delta w}_{\text{"linearization"}} + \underbrace{\frac{1}{2} \Delta w \frac{\partial^2 \mathcal{L}}{\partial w^2} \Delta w}_{\text{"non-linear part"}} + \dots \\ &= \mathcal{L}(w) + g^T \Delta w + \frac{1}{2} \Delta w H \Delta w + \dots \end{aligned}$$

$$g = \frac{\partial \mathcal{L}}{\partial w} \quad \text{"gradient"} \quad \text{vector in } \mathbb{R}^d$$

$$H = \frac{\partial^2 \mathcal{L}}{\partial w^2} \quad \text{"Hessian"} \quad \text{matrix in } \mathbb{R}^{d \times d}$$

Second-order optimization: Newton's method

Take the Taylor expansion to second-order:

$$\mathcal{L}(w + \Delta w) \approx \mathcal{L}(w) + g^T \Delta w + \frac{1}{2} \Delta w^T H \Delta w$$

and minimize the RHS with respect to Δw

Take derivative and set to zero $g + H \Delta w = 0$

$$\Rightarrow \boxed{\Delta w = -H^{-1}g} \quad \text{Newton's method}$$

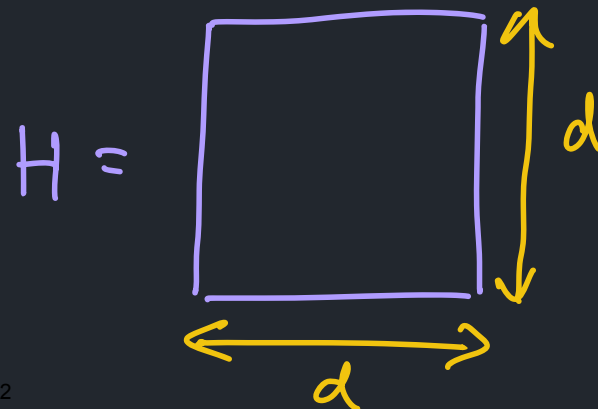
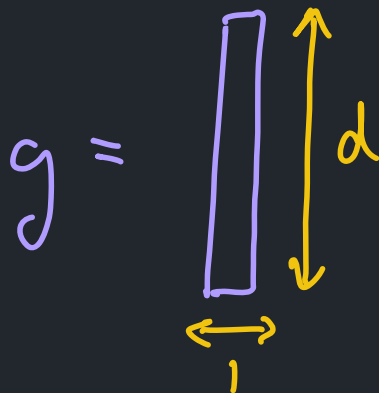
"pre-condition the gradient with the inverted Hessian"

Second-order optimization: Problems w/ Newton

Newton's method $\left| \Delta w = -H^{-1}g \right|$ "pre-condition the gradient with the inverted Hessian"

Problems with Newton's method

- might converge to local max
 - can fix with "cubic regularization"
- d parameters $\Rightarrow$ Hessian is $d \times d$
 - too expensive even for "small" networks



Composite optimization: the GN decomposition

Gauss! Newton!

Suppose we have a "composite" objective $\mathcal{L} = l \circ f$
— e.g. error l composed with neural net f

$$\left\{ \begin{array}{l} \frac{\partial \mathcal{L}}{\partial w} = \frac{\partial l}{\partial f} \cdot \frac{\partial f}{\partial w} \quad \leftarrow \text{chain rule} \\ \frac{\partial^2 \mathcal{L}}{\partial w^2} = \frac{\partial f}{\partial w} \cdot \frac{\partial^2 l}{\partial f^2} \cdot \frac{\partial f}{\partial w} + \frac{\partial l}{\partial f} \frac{\partial^2 f}{\partial w^2} \quad \leftarrow \text{product rule + chain rule} \end{array} \right.$$

we call the second result the "Gauss-Newton" decomposition of the Hessian

Composite optimization: the GN method

Given composite loss function $\mathcal{L} = L \circ f$

$$\frac{\partial^2 \mathcal{L}}{\partial w^2} = \frac{\partial f}{\partial w} \cdot \frac{\partial^2 L}{\partial f^2} \cdot \frac{\partial f}{\partial w} + \frac{\partial L}{\partial f} \frac{\partial^2 f}{\partial w^2}$$

↖ full Hessian H

↖ curvature of error

↖ curvature of model

- ① for square loss, $L = \frac{1}{2} (f - y)^2$, we have $\frac{\partial^2 L}{\partial f^2} = 1$
- ② ignore the curvature of the model 😞

⇒ Newton's method becomes

$$\Delta w = - \left[\frac{\partial f}{\partial w} \frac{\partial f}{\partial w} \right]^{-1} g$$

Gauss
Newton
method

Composite optimization: Problems w/ GN method

$$\boxed{\Delta w = - \left[\frac{\partial f}{\partial w} \frac{\partial f}{\partial w} \right]^{-1} g} \quad \begin{array}{l} \text{Gauss} \\ \text{Newton} \\ \text{method} \end{array}$$

- ① requires computing extra derivatives $\frac{\partial f}{\partial w}$
- ② is it safe to ignore curvature of the model?

$$\frac{\partial^2 \mathcal{L}}{\partial w^2} = \frac{\partial f}{\partial w} \cdot \frac{\partial^2 \mathcal{L}}{\partial f^2} \cdot \frac{\partial f}{\partial w} + \cancel{\frac{\partial \mathcal{L}}{\partial f} \frac{\partial^2 f}{\partial w^2}}$$

↑ full Hessian H ↑ curvature of error ↑ curvature of model

First-order optimization: Steepest descent

Take the Taylor expansion:

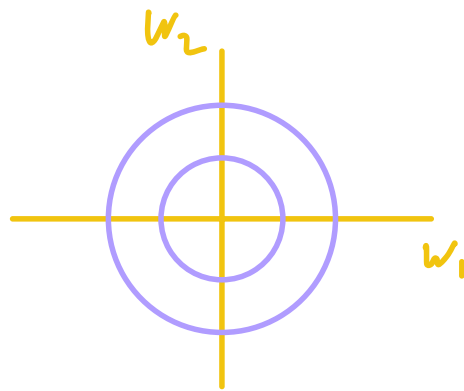
$$\mathcal{L}(w + \Delta w) = \mathcal{L}(w) + g^T \Delta w + \underbrace{\frac{1}{2} \Delta w^T H \Delta w + \dots}_{\text{non-linear part}}$$

$\hookrightarrow$ model with $\frac{\lambda}{2} \|\Delta w\|^2$

$$\text{model} \left\{ \left[\mathcal{L}(w + \Delta w) \approx \mathcal{L}(w) + g^T \Delta w + \frac{\lambda}{2} \|\Delta w\|^2 \right] \right.$$

First-order optimization: ℓ_2 steepest descent

$$\text{model} \left\{ \left[\mathcal{L}(w + \Delta w) \approx \mathcal{L}(w) + g^\top \Delta w + \underbrace{\frac{\lambda}{2} \|\Delta w\|_2^2}_{\text{Euclidean norm}} \right] \right.$$



} Euclidean balls

Euclidean norm

Minimize RHS of model wrt Δw

$\Rightarrow$ differentiate and set derivative to zero

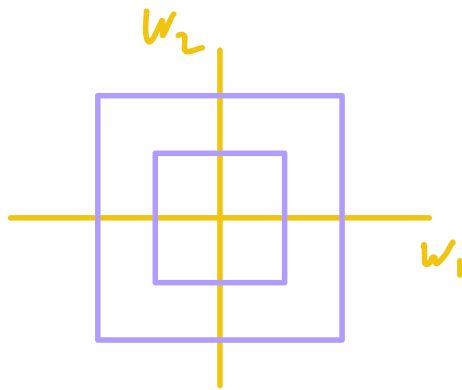
$$\Rightarrow g + \lambda \Delta w = 0$$

$$\Rightarrow \boxed{\Delta w = -\frac{1}{\lambda} g}$$

vanilla
gradient
descent!

First-order optimization: ℓ_∞ steepest descent

$$\text{model} \left\{ \left[\mathcal{L}(w + \Delta w) \approx \mathcal{L}(w) + g^\top \Delta w + \underbrace{\frac{\lambda}{2} \|\Delta w\|_\infty^2}_{\text{infinity norm}} \right] \right.$$



} infinity balls

infinity norm

Minimize RHS of model wrt Δw

$\Rightarrow$ <do homework>

$$\Rightarrow \left[\Delta w = -\frac{\|g\|_1}{\lambda} \text{sign}(g) \right]$$

sign
gradient
descent!

First-order optimization: General steepest descent

$$\text{model} \left\{ \boxed{\mathcal{L}(w + \Delta w) \approx \mathcal{L}(w) + \underbrace{g^T \Delta w + \frac{\lambda}{2} \|\Delta w\|^2}_{\text{general norm}}} \right.$$

minimizing the right hand side with respect to Δw has a "dual formulation"

$$\argmin_{\Delta w} \left[g^T \Delta w + \frac{\lambda}{2} \|\Delta w\|^2 \right] \equiv \frac{\|g\|^+}{\lambda} \quad \argmax_{t: \|t\|=1} g^T t$$

step size

step direction

where $\|\cdot\|^+$ is the "dual norm" to $\|\cdot\|$

Heuristics for Scaling

How large should the weight updates be?

We want the "Goldilocks" update size...

... not too big, not too small

But always ask: in which norm?

observation a neural net is built out of weight matrices

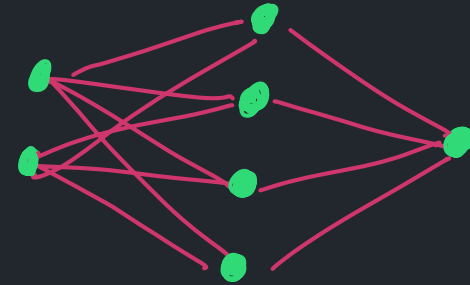
perhaps we could try a matrix norm?

Frobenius norm
e.g. spectral norm
nuclear norm

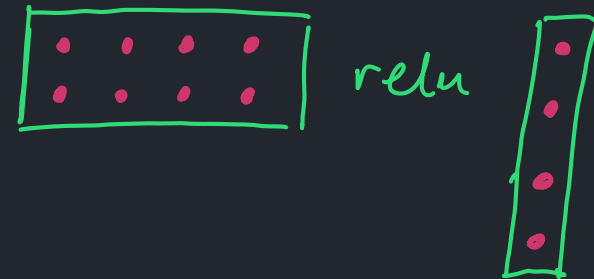
⋮

Perspectives on neural computation

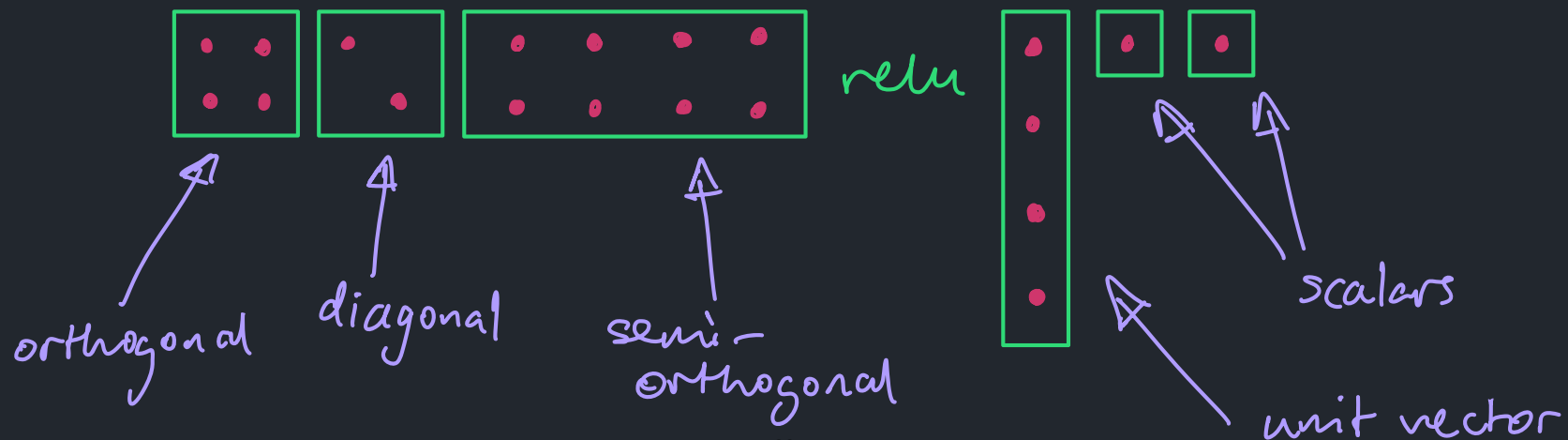
① NEURAL PERSPECTIVE



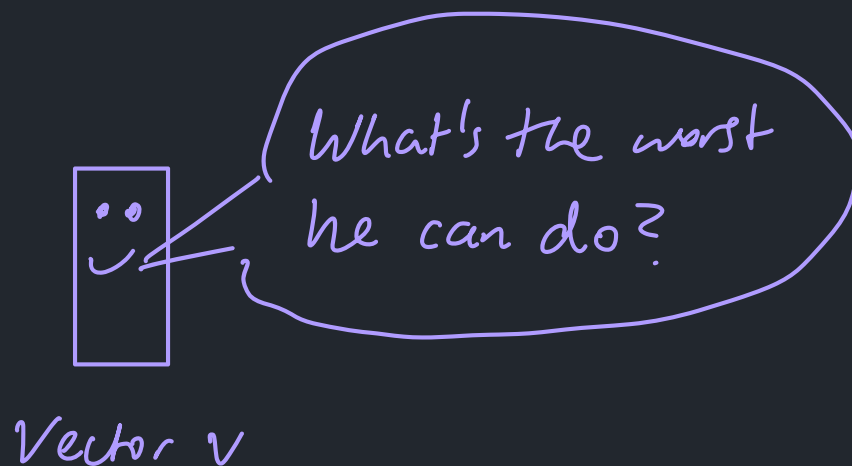
② TENSOR PERSPECTIVE



③ SPECTRAL PERSPECTIVE using SVDs



The spectral norm



Spectral norm $\|M\|_* = \max_{v \neq 0} \frac{\|Mv\|_2}{\|v\|_2}$

Answers question: how much can a matrix scale up the Euclidean norm of a vector?

FACT spectral norm $\equiv$ largest singular value

The RMS-RMS operator norm



Matrix M

Grrr...



Vector $v \in \mathbb{R}^d$

What's the worst he can do?

Equip vectors in $\mathbb{R}^d$ with the RMS norm $\|\cdot\|_{\text{rms}} = \frac{1}{\sqrt{d}} \|\cdot\|_2$

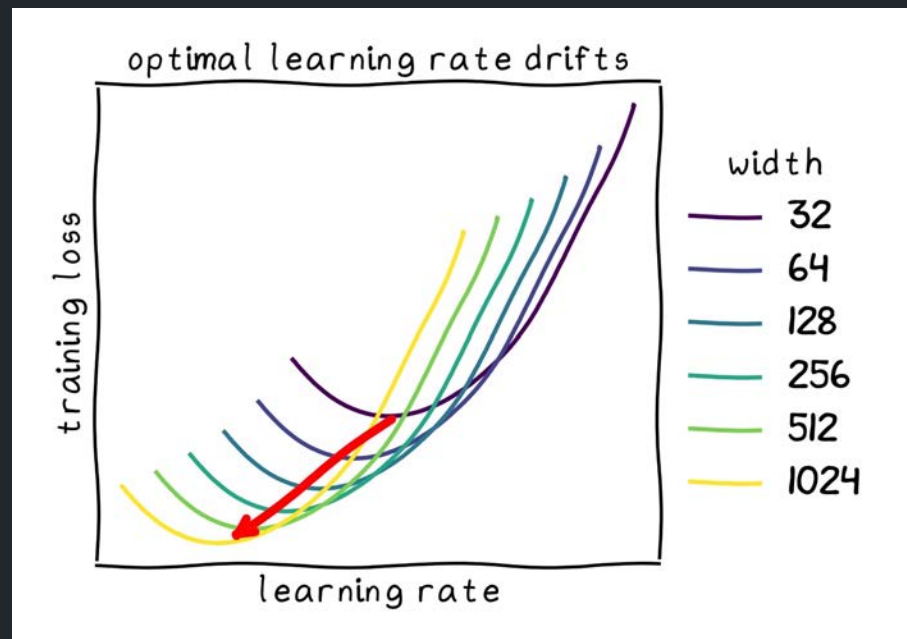
RMS-RMS operator norm $\|M\|_{\text{rms-rms}} = \max_{v \neq 0} \frac{\|Mv\|_{\text{rms}}}{\|v\|_{\text{rms}}}$

Interpretation $\|\cdot\|_{\text{rms-rms}}$ constrains how much a matrix can change the RMS norm of its input

Exercise show that $\| \cdot \|_{\text{rms-rms}} = \sqrt{\frac{d_{\text{in}}}{d_{\text{out}}}} \| \cdot \|_*$

Spectrally controlled weight updates

recall our scaling woes



© @kellerjordan0 on X. All rights reserved. This content is excluded from our Creative Commons license. For more information, see

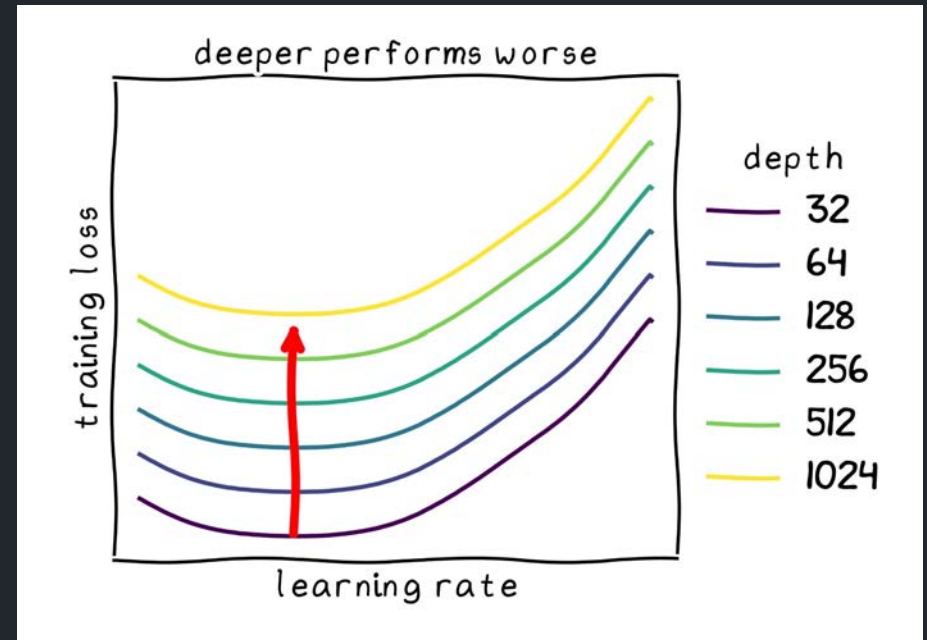
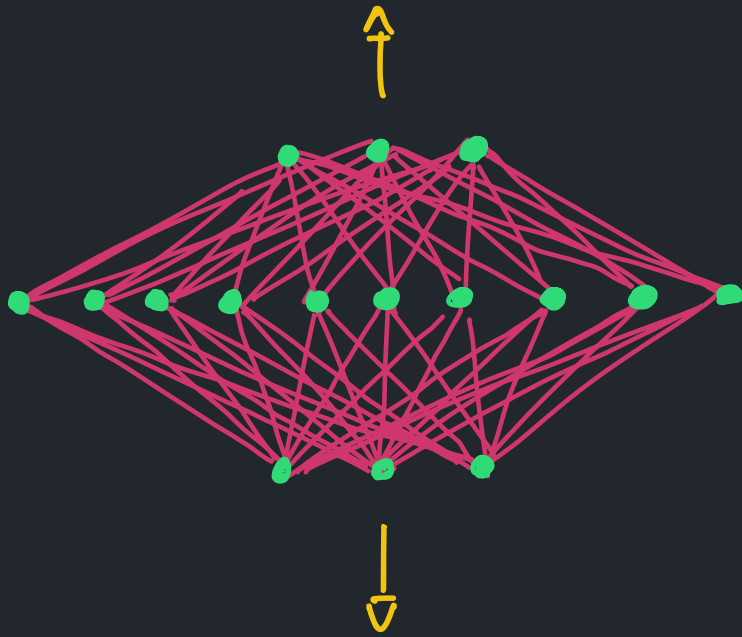
<https://ocw.mit.edu/help/faq-fair-use/>

Claim: to remove drift in optimal learning rate as width is varied, for all layers $l=1, \dots, L$, do:

- ① initialize weights so that $\|W_l\|_{\text{rms-rms}} \sim 1$.
- ② scale updates so that $\|\Delta W_l\|_{\text{rms-rms}} \sim 1$.

See homework!

Depth scaling



© @kellerjordan0 on X. All rights reserved. This content is excluded from our Creative Commons license. For more information, see <https://ocw.mit.edu/help/faq-fair-use/>

the trick seems to be to parameterize your residual block the "right" way

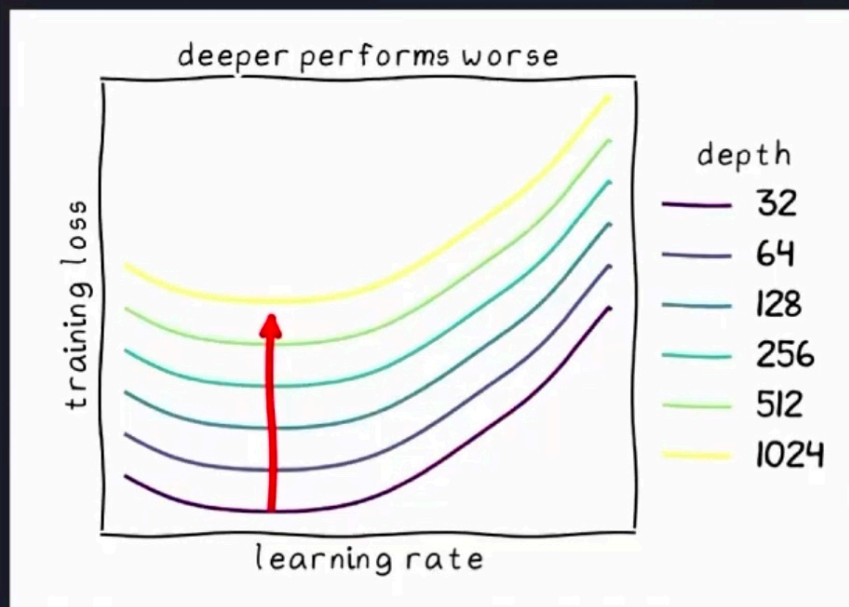
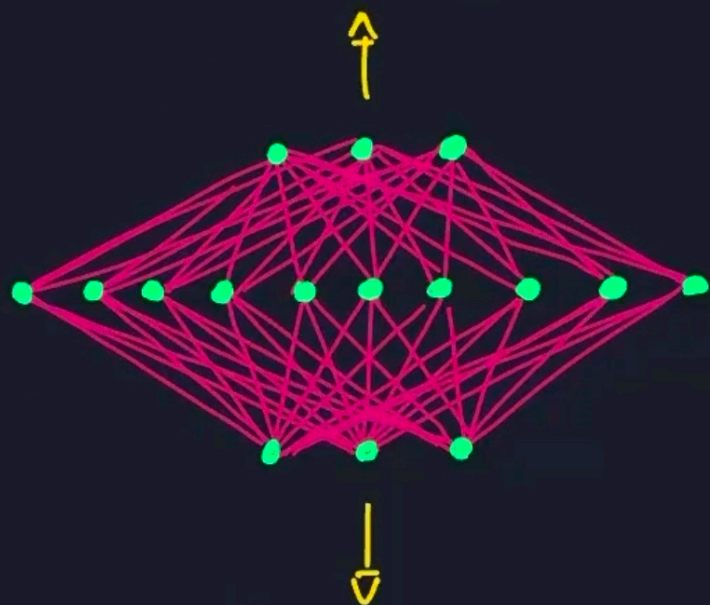
recall that $\lim_{L \rightarrow \infty} \left(1 + \frac{x}{L}\right)^L = \exp(x)$

so build your residual block like

$$x \rightarrow x + \frac{1}{L} \text{layer}(x) ?$$

needs more research

Depth scaling



the trick seems to be to parameterize your residual block the "right" way

recall that $\lim_{L \rightarrow \infty} \left(1 + \frac{x}{L}\right)^L = \exp(x)$

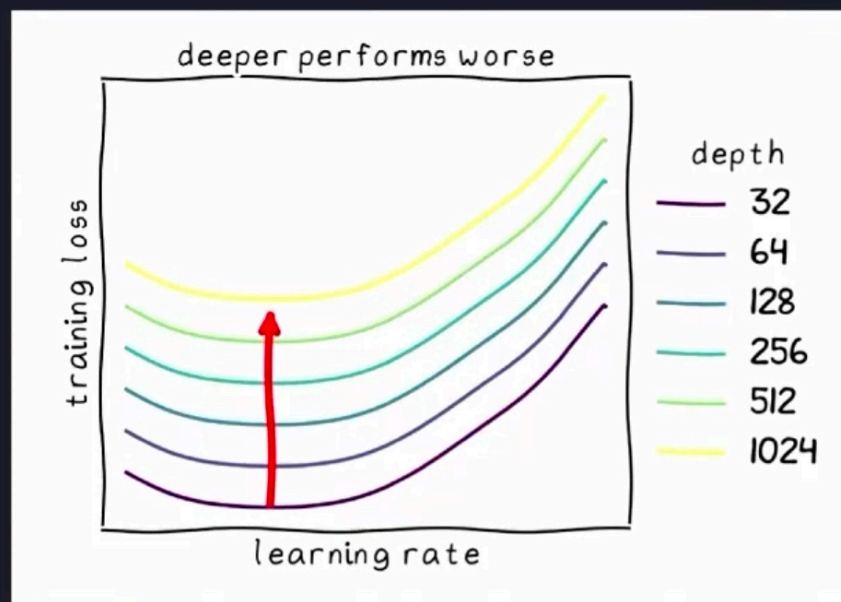
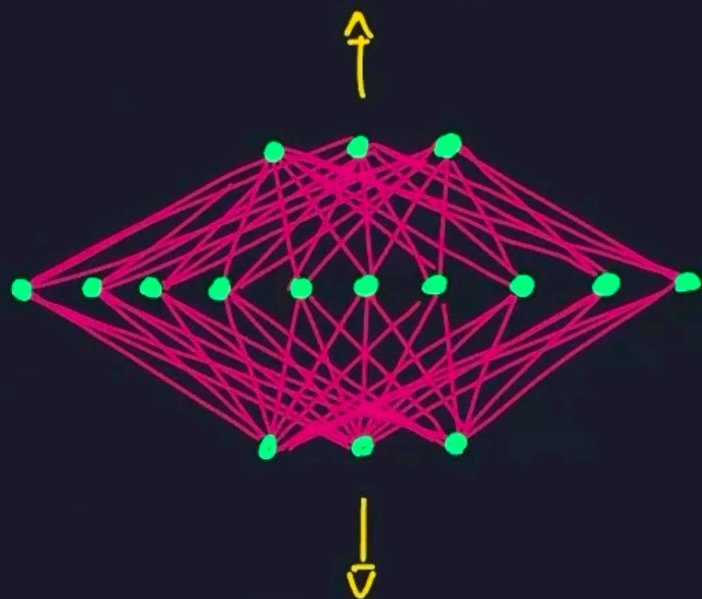
call L. So intuitively, you

can think of a residual network

$x \rightarrow x + \frac{1}{L} \text{Block}(x)$

(needs more research)

deeper performs worse



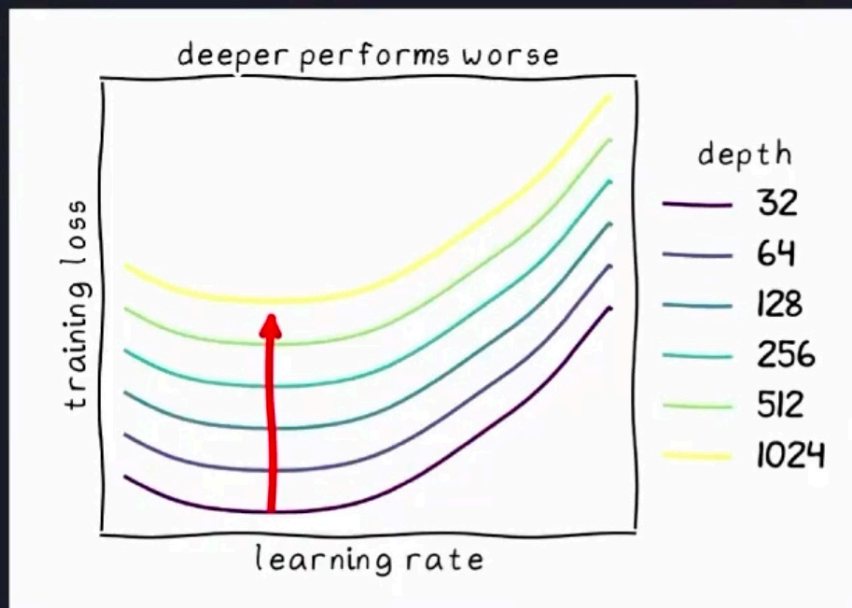
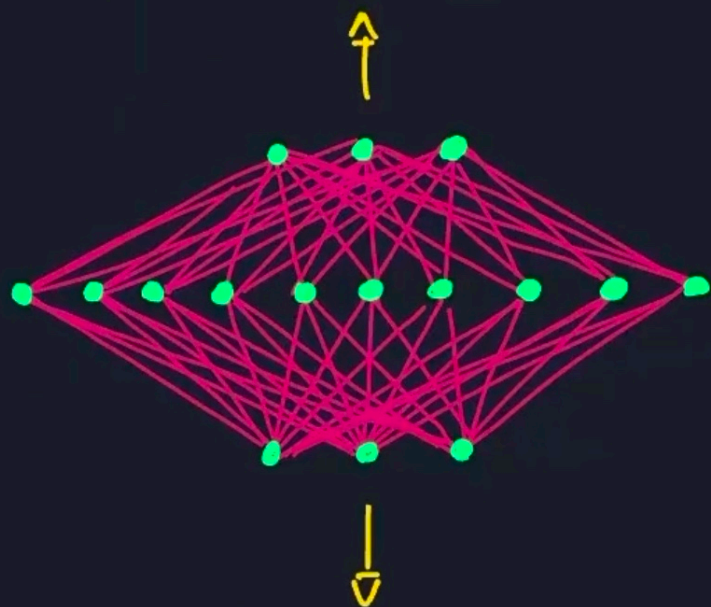
the trick seems to be to parameterize your residual block the "right" way

recall that $\lim_{L \rightarrow \infty} \left(1 + \frac{x}{L}\right)^L = \exp(x)$

but that really means composing it with itself L times.

$$\left[x + \frac{1}{L} \text{Block}(x) \right]^L$$

(needs more research)



the trick seems to be to parameterize your residual block the "right" way

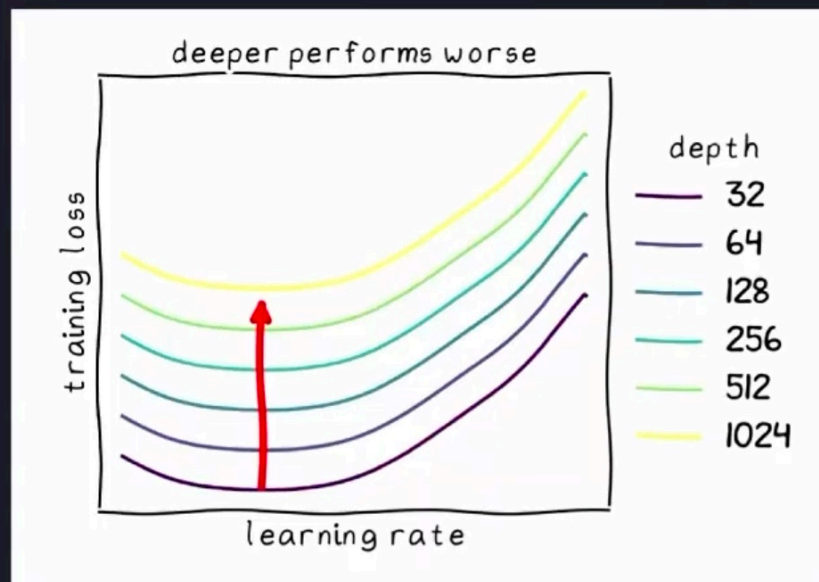
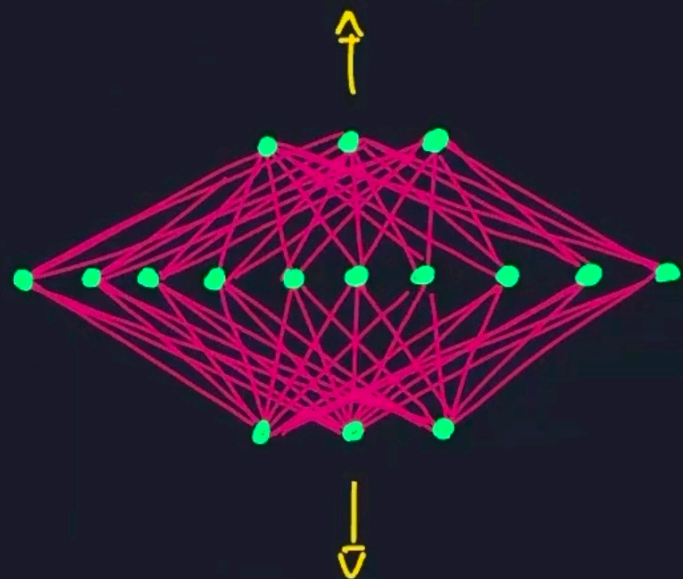
recall that $\lim_{L \rightarrow \infty} \left(1 + \frac{x}{L}\right)^L = \exp(x)$

see that a standard transformer block actually doesn't put a 1

$\left[x + \frac{1}{L} \text{Block}(x) \right]^L$

(needs more research)

depth scaling



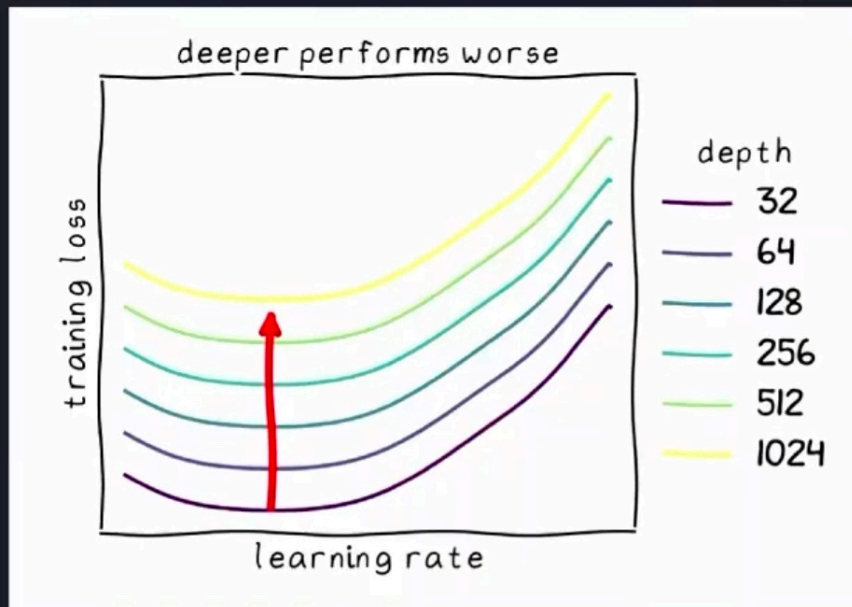
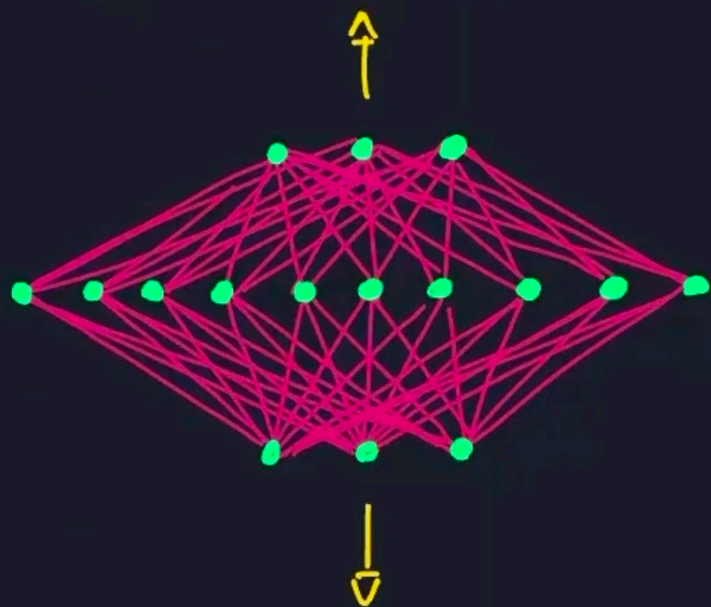
the trick seems to be to parameterize your residual block the "right" way

recall that $\lim_{L \rightarrow \infty} \left(1 + \frac{x}{L}\right)^L = \exp(x)$

are somehow incoherent
and random with respect
to $\sqrt{K}$ Block(x)

(needs more research)

Depth scaling



the trick seems to be to parameterize your residual block the "right" way

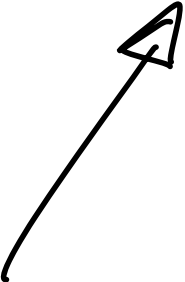
recall that $\lim_{L \rightarrow \infty} \left(1 + \frac{x}{L}\right)^L = \exp(x)$

But it's really a thing, which is part of a transformer code

$x + \frac{1}{\sqrt{K}} \text{Block}(x)$

(needs more research)

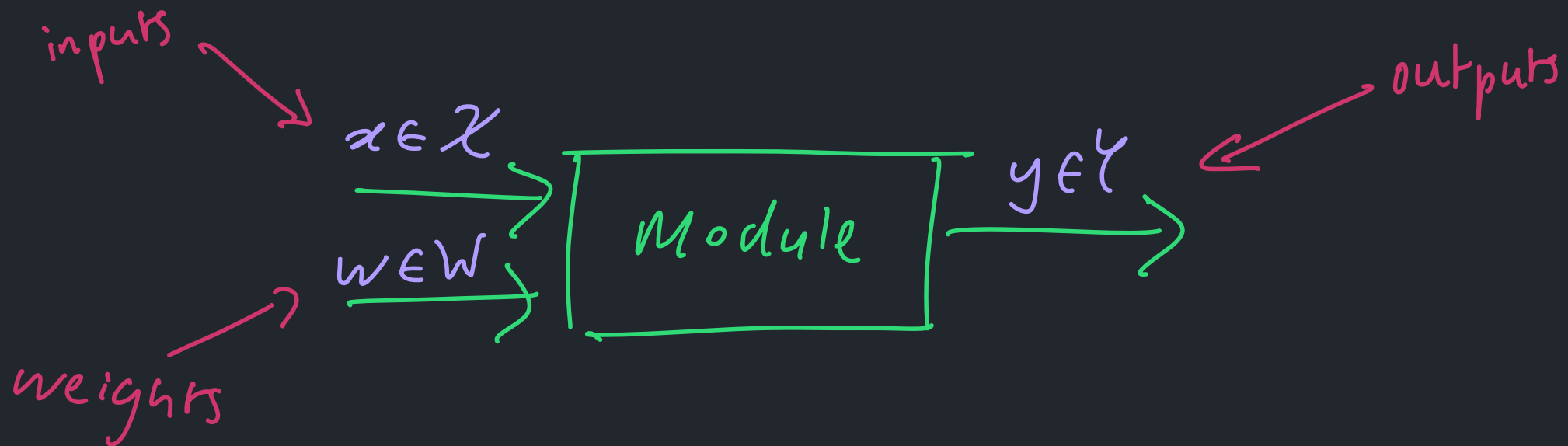
Building a Theory : Modularization


my research, so be skeptical

Building a modular theory

IDEA: if you want an optimization theory that handles complicated neural networks

... build the theory with the neural net



Write a library of "atomic modules"

Definition module M

$M.$ forward

$M.$ backward

Think about this as writing
your own deep learning package,

Write a library of "atomic modules"

Definition module M

$M.$ forward $W \times \mathcal{X} \rightarrow \mathcal{Y}$

$M.$ backward $\mathcal{Y} \times W \times \mathcal{X} \rightarrow W \times \mathcal{X}$

$M.$ norm $W \rightarrow \mathbb{R}$

Linear

Embedding

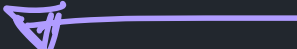
Conv2D

ReLU

} all with hand-specified
forward, backward and norms

Write combination rules

e.g. module composition $M = M_2 \circ M_1$

M .forward  just compose M_2 .forward
with M_1 .forward

M .backward



M .norm



do the chain rule

how should we combine
 M_2 .norm and M_1 .norm?

write a library of "atomic modules"

Definition module M

M . forward

M . backward

M . norm : $\mathbb{R}^n \rightarrow \mathbb{R}$

ReLU



Linear



RMS - RM

Conv2D



Embedding

is this RMS to
RMS operator norm.

Write combination rules

.g. module composition $M = M_2 \circ M_1$

M .forward $\leftarrow$ function composition

M .backward

M .norm

chain rule

Do you remember we were
talking about steepest descent?

how to combine

M_1 .norm and M_2 .norm

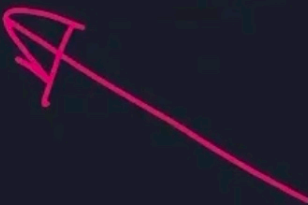
WRITE COMBINATION RULES

e.g. module composition $M = M_2 \circ M_1$

$M.forward$  function composition

$M.backward$

$M.norm$

 chain rule

it automatically gives them
a norm on the composed thing.

 how to combine $M_1.norm$ and $M_2.norm$

References

-  Feature Learning in Infinite Width Neural Networks
Yang & Hu (2020)
-  Infinite Limits of Multi-Head Transformer Dynamics
Bordelon, Chaudhury & Pehlevan (2024)
-  Scalable Optimization in the Modular Norm
Large et al (2024)
-  Universal Majorization - Minimization Algorithms
Streeter (2023)

MIT OpenCourseWare

<https://ocw.mit.edu>

6.7960 Deep Learning

Fall 2024

For information about citing these materials or our Terms of Use, visit: <https://ocw.mit.edu/terms>